Design a Car Rental System

Problem Statement
Flow and Requirement Classification
1. Vehicle, VehicleStatus and VehicleType
2. VehicleInventoryManager
3. Reservation
4. ReservationManager
5. Bill
6. Payment
7. Store
Class Diagram
Implementation

Resources

- 9. LLD of Car Rental System (Hindi) | ZoomCar Low Level Design | System Design Interview Question
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/carrental · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Problem Statement

Design a detailed low-level model for a Car Rental System, including all relevant classes and design patterns covered earlier.

Flow and Requirement Classification

- User comes to care rental system.
- Select a particular Store based on its Location.
- Search for Vehicles based on vehicle type(like "Four Vehicle", "Two Wheeler" etc.) and its availability.
- Reserve the selected vehicle.
  a. Should handle concurrency, as 2 users try to reserve the same vehicle for same time.
- Start the trip.
- Submit the vehicle.
- Bill is generated against the reservation.
  a. We can have multiple bill strategies for generating the bill, like daily Bill strategy or hourly bill strategy etc.

- Payment is made against the Bill.
    a. We can have multiple payment strategies for processing the payment like UPI, Card etc.

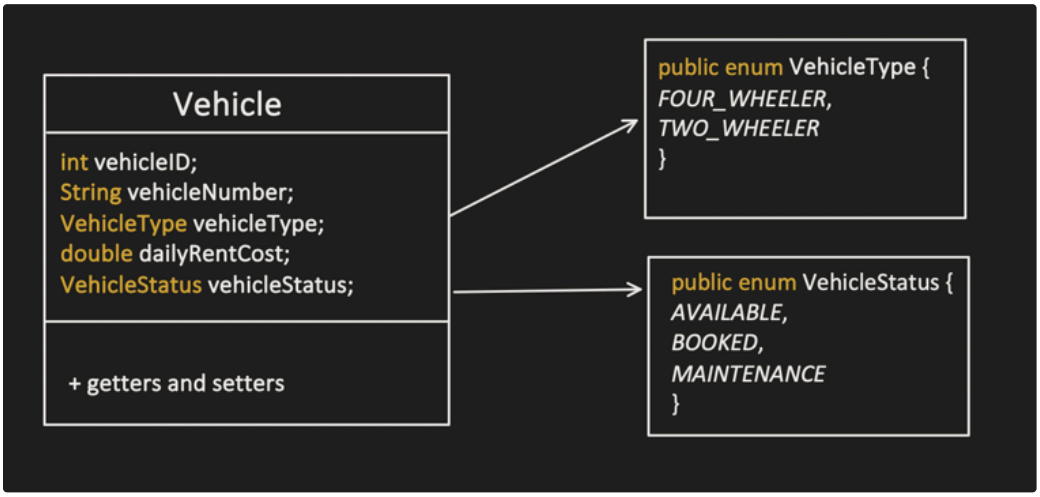
Based on above understanding, we identified the below major objects involved in this design:

- Vehicle
- Store
- Reservation
- User
- Bill
- Payment

<u>Lets start creating the UML:</u>

1. `Vehicle`, `VehicleStatus` and `VehicleType`

- Vehicle class represents the basic building block of the Car Rental System.
- It comprises all the necessary details about the vehicle that is available for renting out, like vehicle number, type, model, odometer readings, date of manufacture and its rental charge.
- `VehicleStatus` and `VehicleType` help track and store more details about the `Vehicle` in use.



Vehicle object is not intelligent, its just a POJO. We need some other intelligent object, which can manage Vehicle inventory, like:

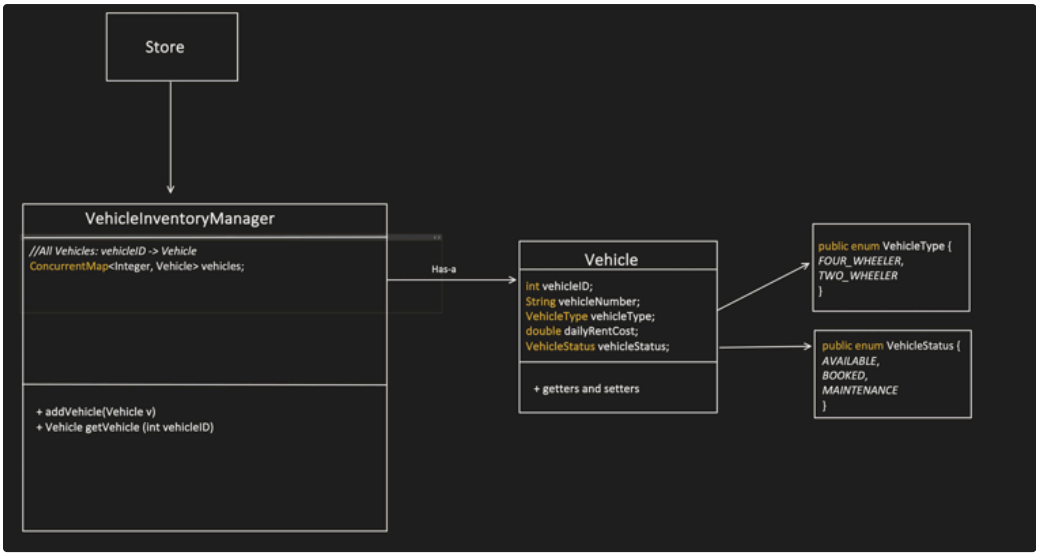
- Maintain overall vehicles list.
- Maintain booked vehicles list.
- Add vehicle
- Remove vehicle
- Check for vehicle availability
  etc.

  That's where VehicleInventoryManager comes into the picture.

  Note:
  each store has their own VehicleInventoryManager, so it is not a singleton class

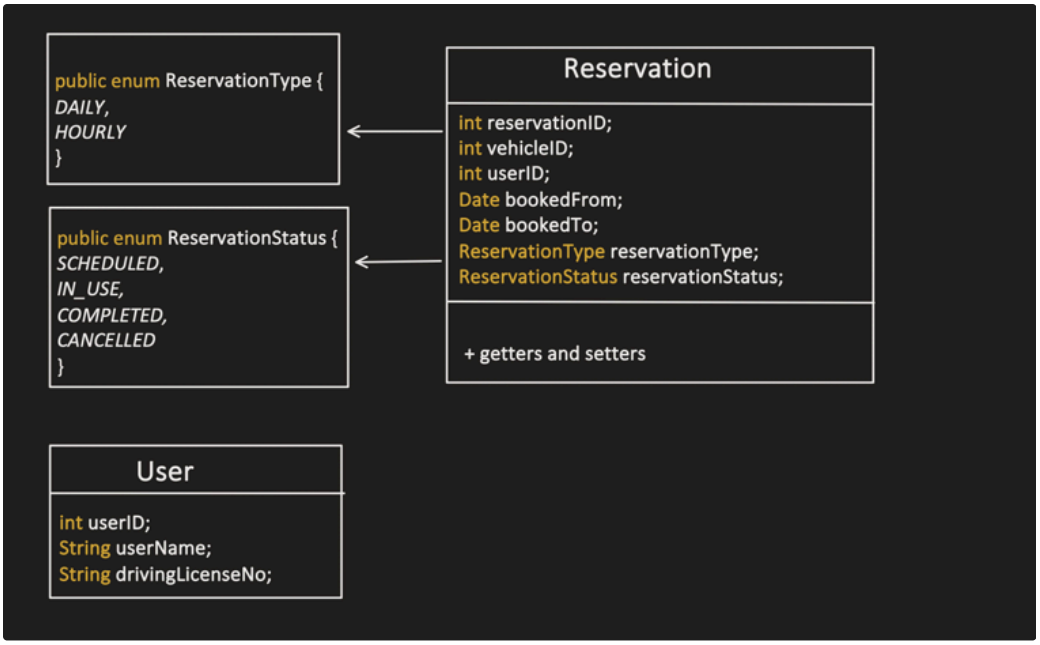
2. `VehicleInventoryManager`

We will enhance the capability of this VehicleInventoryManager later when Reservation capability will get added.



3. `Reservation`

- Reservation, does not need full Vehicle and User details, so we can keep their ids itself in it.



Now, Reservation is not smart enough to handle capabilities like:

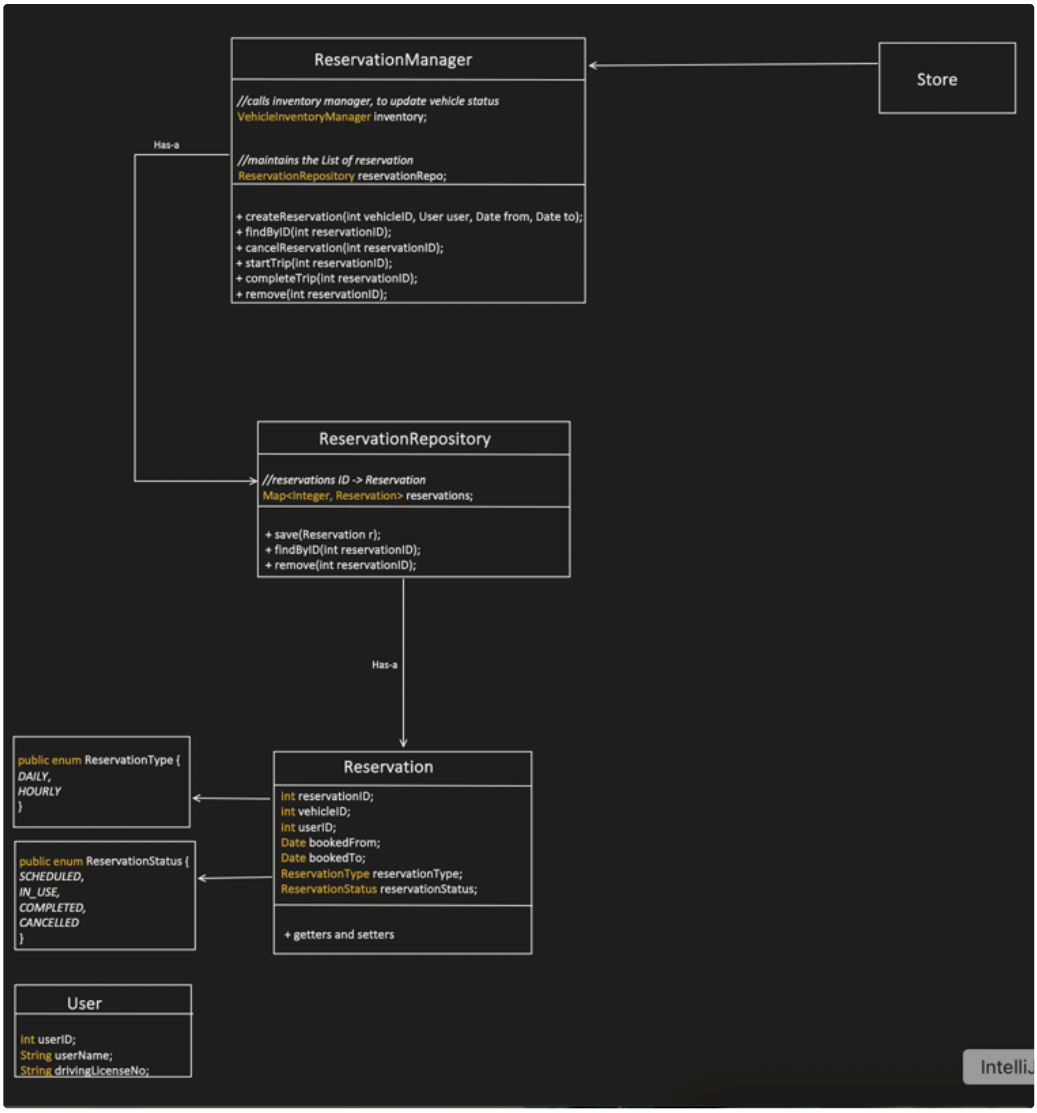
- Maintaining the List of Reservations.
- Fetch particular reservation details.
- Creation of reservation, changing the reservation status.
- Cancellation of reservation, changing the reservation status.
- Remove the reservation

Etc.

That's where ReservationManager comes into the picture.

Note:
each store has their own ReservationManager, so it is not a singleton class

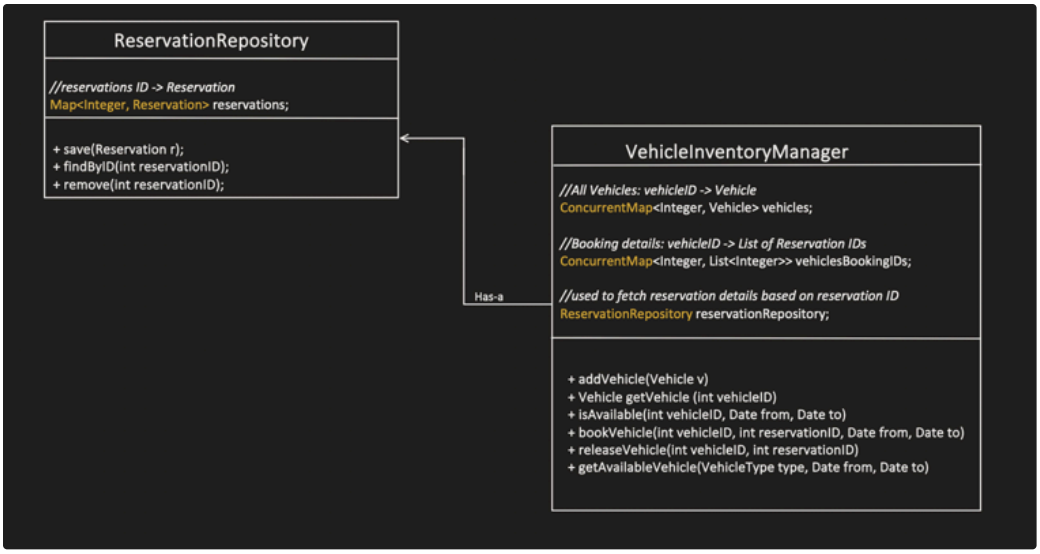
4. `ReservationManager`



Need to handle:

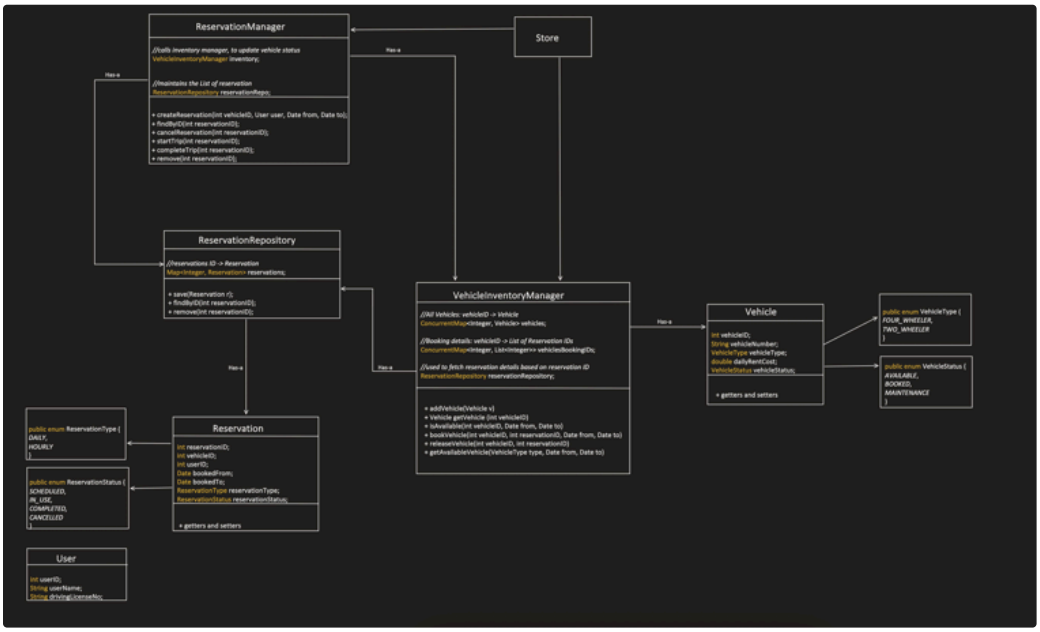
Create Reservation should only be successful, when vehicle is available on that particular date.

Whats the right place to hold the vehicle booking information. Its "VehicleInventoryManager"



Note:

I created a ReservationRepository to break the circular dependency between ReservationManager and VehicleInventoryManager.



Another thing which need to be handle is: Concurrency

2 important things to consider while handling the concurrency:

1st: Collections like List of vehicles, List of Booked vehicles should be able to handle concurrency. So that our underlying data structure should be consistent when multiple threads

try to read or write in it.

2nd: Need capability, to put lock per Vehicle, this is needed for achieve atomicity.

For example: assume both thread1 and thread2 running concurrently, below scenario will cause an issue:

Thread1:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Thread2:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- Try to reserve this particular vehicle (V1), change the status of vehicle
- Update the List of Booked vehicles

Now, even though our collections are concurrent (only 1 thread can touch it at a time), still above scenario will cause an issue and both the

thread will be able to book the same vehicle for same slot.

To handle it, we need to bring lock per vehicle:

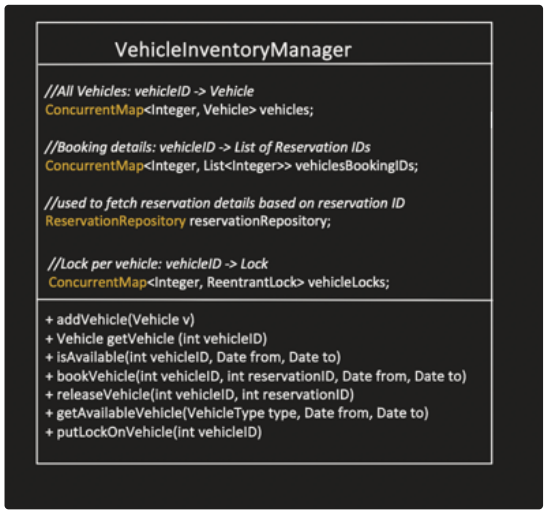
Thread1:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle : V1
- **Put a lock on V1**
- **Check for availability again**
- Try to reserve this particular vehicle(V1), change the status of vehicle
- Update the List of Booked vehicles
- **Release lock**

Thread2:

- Check for vehicle availability, touches the List of vehicles
- Select a particular vehicle: V1
- **Put a lock on V1**
- **Check for availability again**
- Try to reserve this particular vehicle, change the status of vehicle
- Update the List of Booked vehicles
- **Release lock**

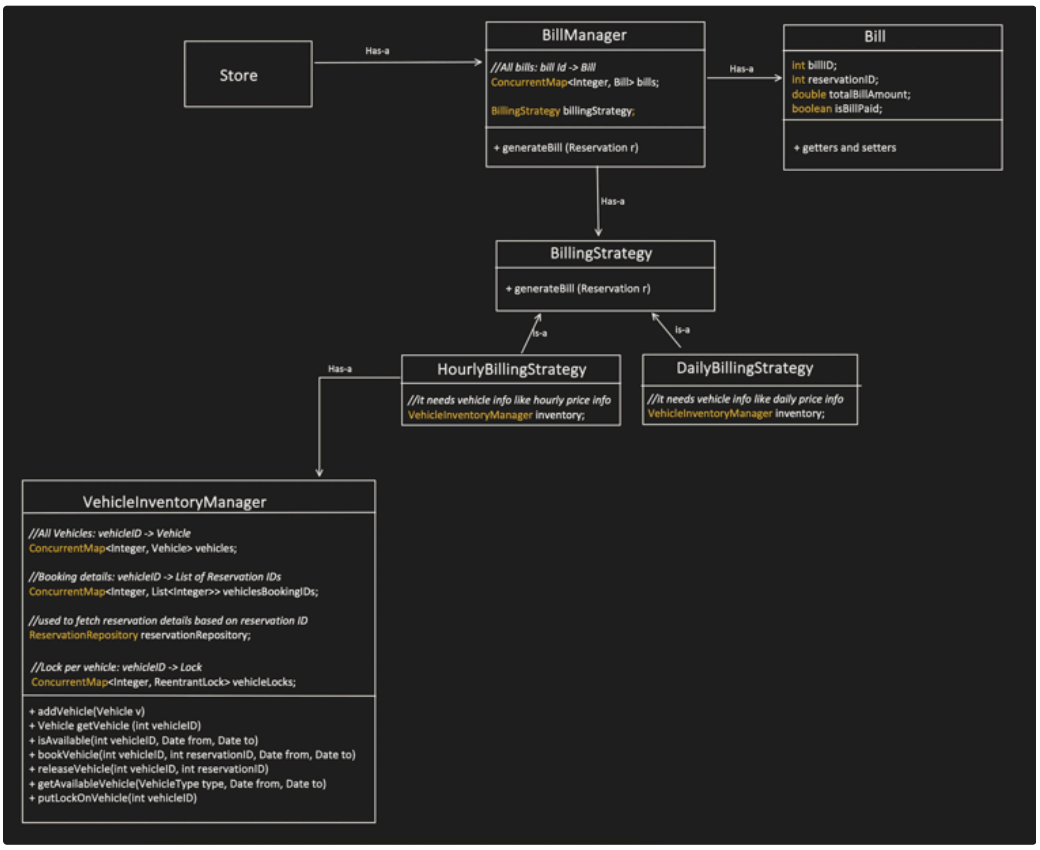
Now, one thread say Thread2 will wait for the lock to release on V1.
When Thread1, release the lock, then Thread2 will proceed and check for availability again, but this time it will not be able to reserve as its not available.



5. `Bill`

- Bill hold info like Reservation id like this bill is associated with which reservation. We don't need full object.
- Also, Bill is just a POJO class which is not smart enough. So we need:
  BillManager, which handles all the bills and their lifecycle.

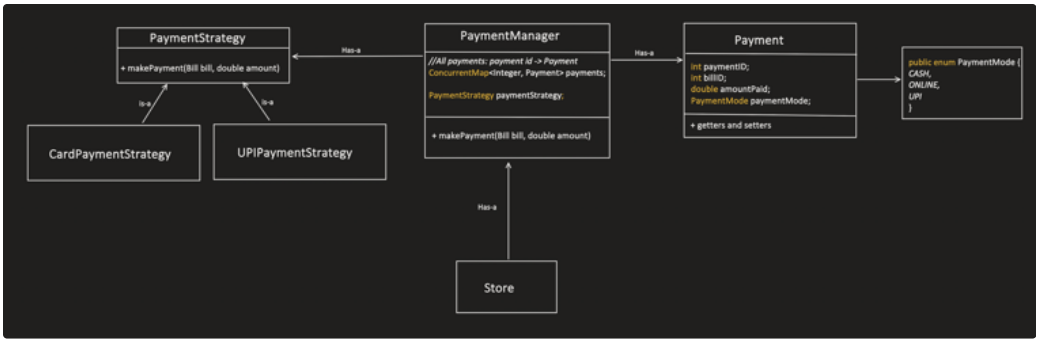
Store, use this BillManager to *generateBill* for the given reservation.



6. `Payment`

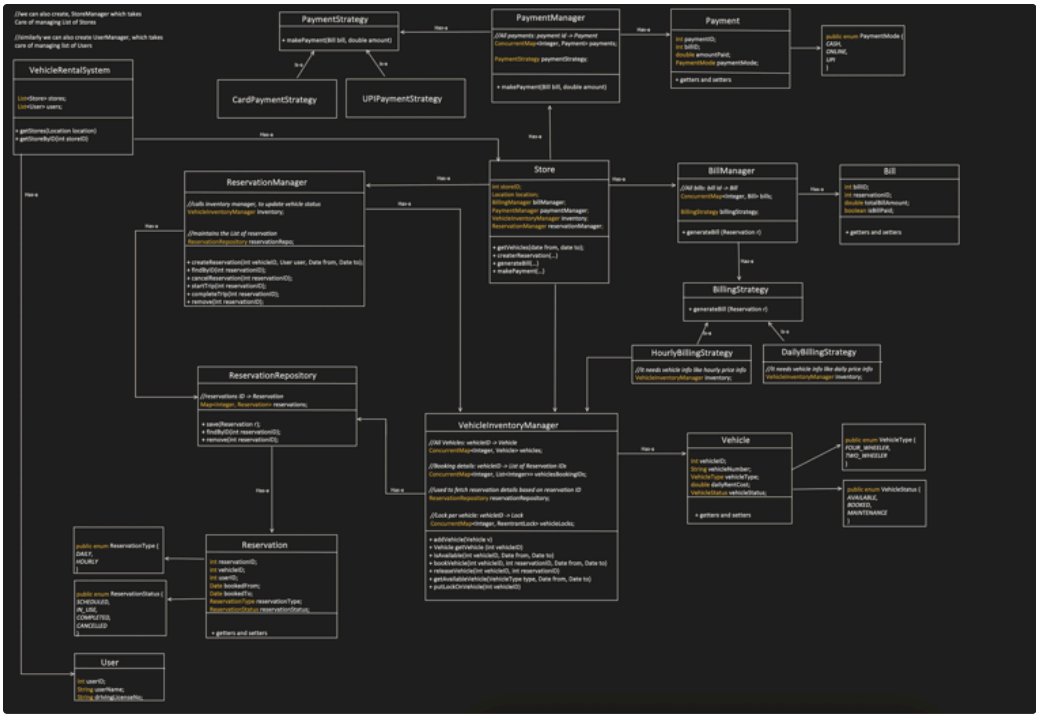
- Payment hold info like Bill id like this payment is associated with which bill. We don't need full bill object.
- Also, Payment is just a POJO class which is not smart enough. So we need: PaymentManager, which handles all the payments and their lifecycle.

  Store, use this PaymentManager to *make payment* for the given bill.



7. `Store`

Class Diagram



Implementation

Refer Code Repository for Executable Code → src/main/java/com/conceptcoding/interviewquestions/carrental · main · shrayansh jain / LLD-LowLevelDesign · GitLab